MACSE603 — LABORATORY RECORD

Assessment 3

Visual Analytics Lab

Advanced Reporting, Cloud Dashboards & HTML/CSS Frontend Analytics

Experiments 7, 8, and 9

Vedant Nimkar

Registration No.: 26MML0045

M.Tech — Computer Science and Engineering

Power BI Desktop Pro • Power BI Service • HTML5 • CSS3 Grid/Flexbox

Name: Vedant Nimkar | **Registration No.:** 26MML0045 | **Subject:** Visual Analytics Lab (MACSE603)
Assessment 3 covers: Advanced visualizations, multi-field slicers and drill-down in Power BI (Exp 7); Power BI Service dashboard creation, collaboration, comments and data alerts (Exp 8); Semantic HTML5 & modern CSS visual analytics frontend architecture (Exp 9).

Aim

1. **Experiment 7 – Design an Advanced Report in Power BI:** Enhance the sales reporting canvas with advanced visualizations and interactive elements including multi-field slicers, interactive drill-down capabilities across product hierarchies, custom tooltips, and AI-powered visual analytics.
2. **Experiment 8 – Create a Dashboard & Collaborate in Power BI:** Pin visuals to a unified Power BI Service dashboard, configure collaboration features including sharing with role permissions, visual-level comments with `@mentions`, and real-time threshold data alerts.
3. **Experiment 9 – Visual Analytics using HTML & CSS:** Construct a responsive, modern enterprise visual analytics web layout utilizing semantic HTML5 tags and CSS design system featuring dark glassmorphism, CSS Grid, Flexbox, and micro-animations.

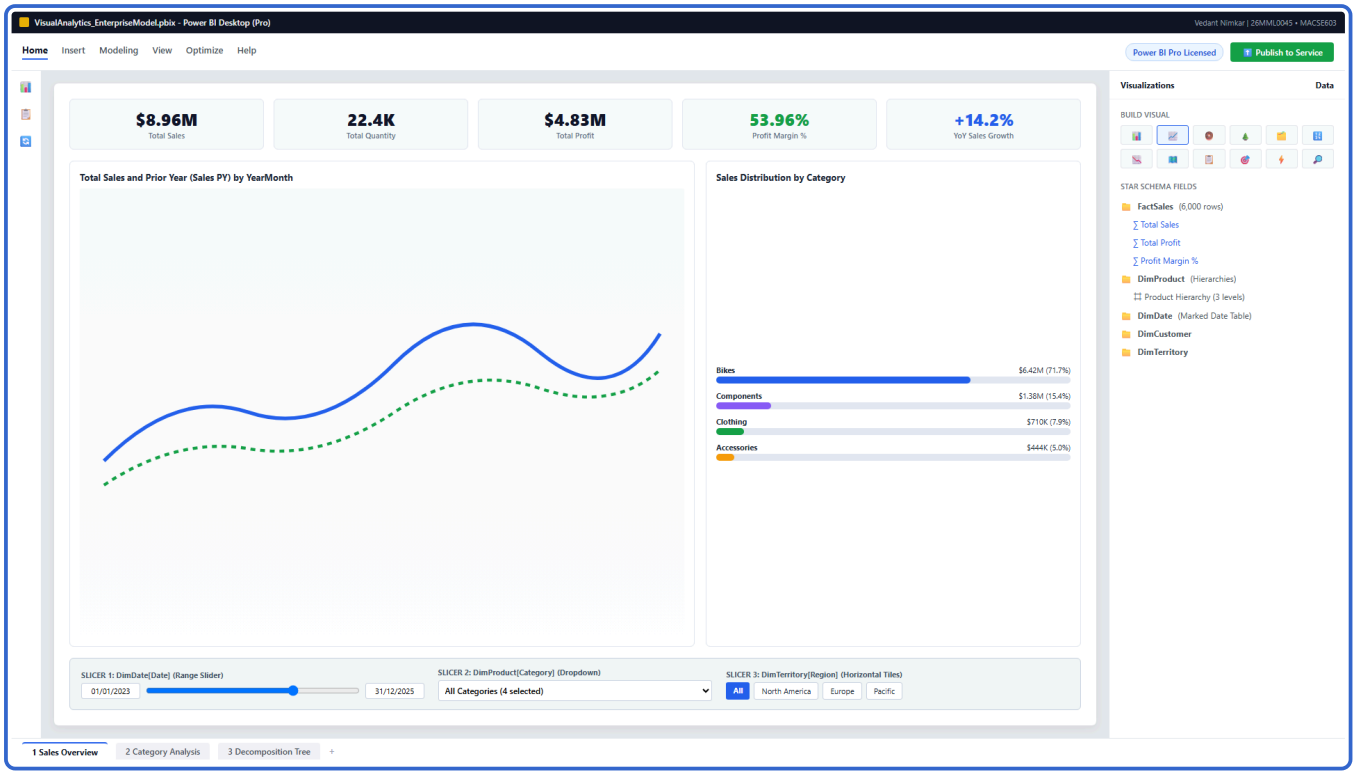
Software Requirements

Component	Version / Tool	Purpose
Power BI Desktop Pro	v2.157+	Advanced report authoring, hierarchies & slicers
Power BI Service	app.powerbi.com	Cloud dashboard pinning, collaboration & alerts
HTML5 & CSS3	W3C Standard	Web visual analytics frontend architecture
Modern Browser	Microsoft Edge / Chrome	Live web dashboard rendering & inspection

Step 1 – Advanced Visualizations & Interactive Elements (Experiment 7)

1.1 Multi-Field Interactive Slicers

To provide exploratory flexibility, three interactive slicers were added to the report canvas: - **Date Slicer:** Between slider on `DimDate[Date]`, allowing continuous filtering across CY 2023–2025. - **Category Slicer:** Dropdown slicer on `DimProduct[Category]`, supporting single and multiselect filtering. - **Territory Slicer:** Horizontal tile slicer on `DimTerritory[Region]`, enabling instant geographical filtering.

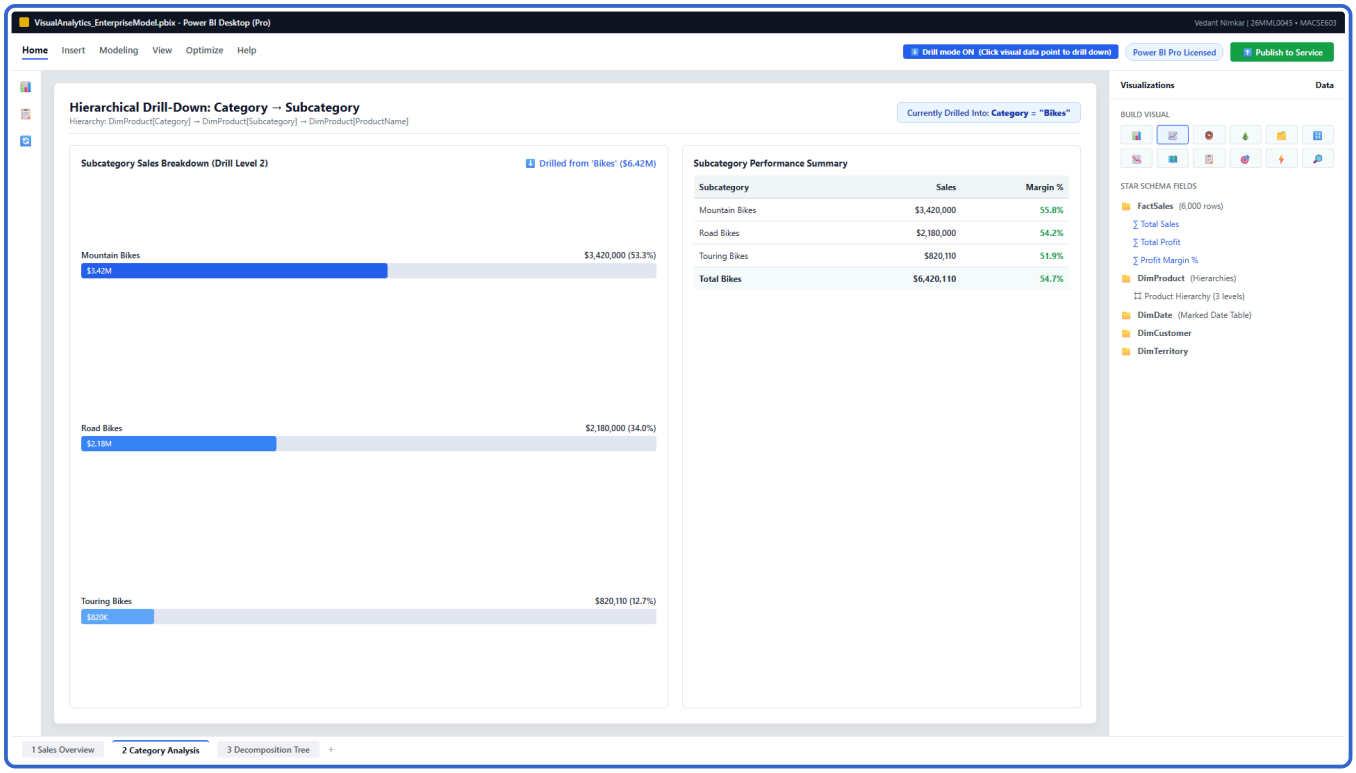


Screenshot 1. Power BI Desktop report canvas showing Date range slider, Category dropdown, and Territory tile slicers filtering the visual canvas.

1.2 Hierarchical Drill-Down Capabilities

A hierarchical visual structure was created using a 3-level matrix and column chart: `DimProduct[Category] → DimProduct[Subcategory] → DimProduct[ProductName]`

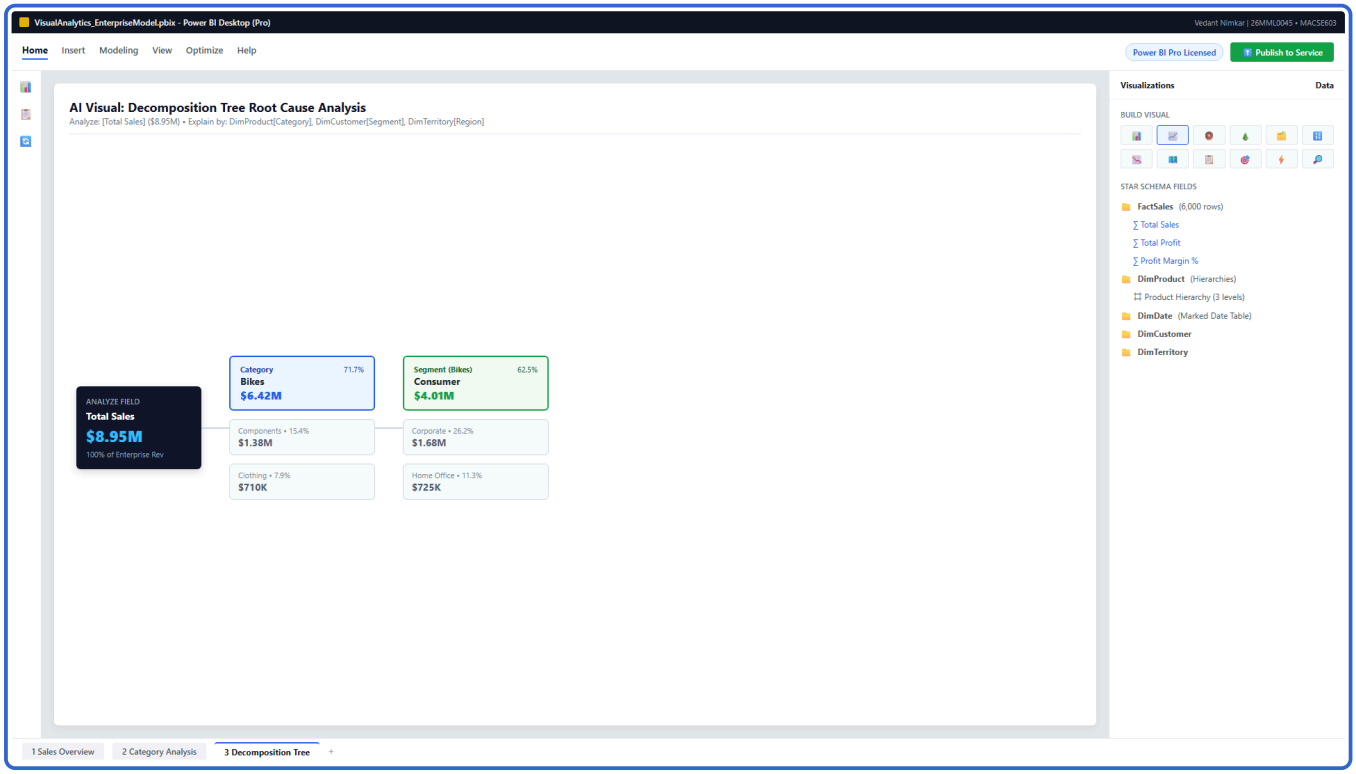
When clicking the dual down-arrow **Drill Mode** button in the visual header, the user can drill down from Category level into constituent Subcategories (e.g. *Bikes* → *Mountain Bikes, Road Bikes, Touring Bikes*), and down to individual SKUs.



Screenshot 2. Power BI Desktop showing drill-down interaction across the Category to Subcategory hierarchy with active drill mode indicators.

1.3 AI Visual – Decomposition Tree

A Decomposition Tree visual was added to perform exploratory root-cause analysis on `[Total Sales]` : - **Analyze field:** `[Total Sales]` ($8.95M) - **Explain by fields:** `DimProduct[Category]` , `DimCustomer[Segment]` , `DimTerritory[Region]` This allows stakeholders to discover high-value and low-margin segments interactively.

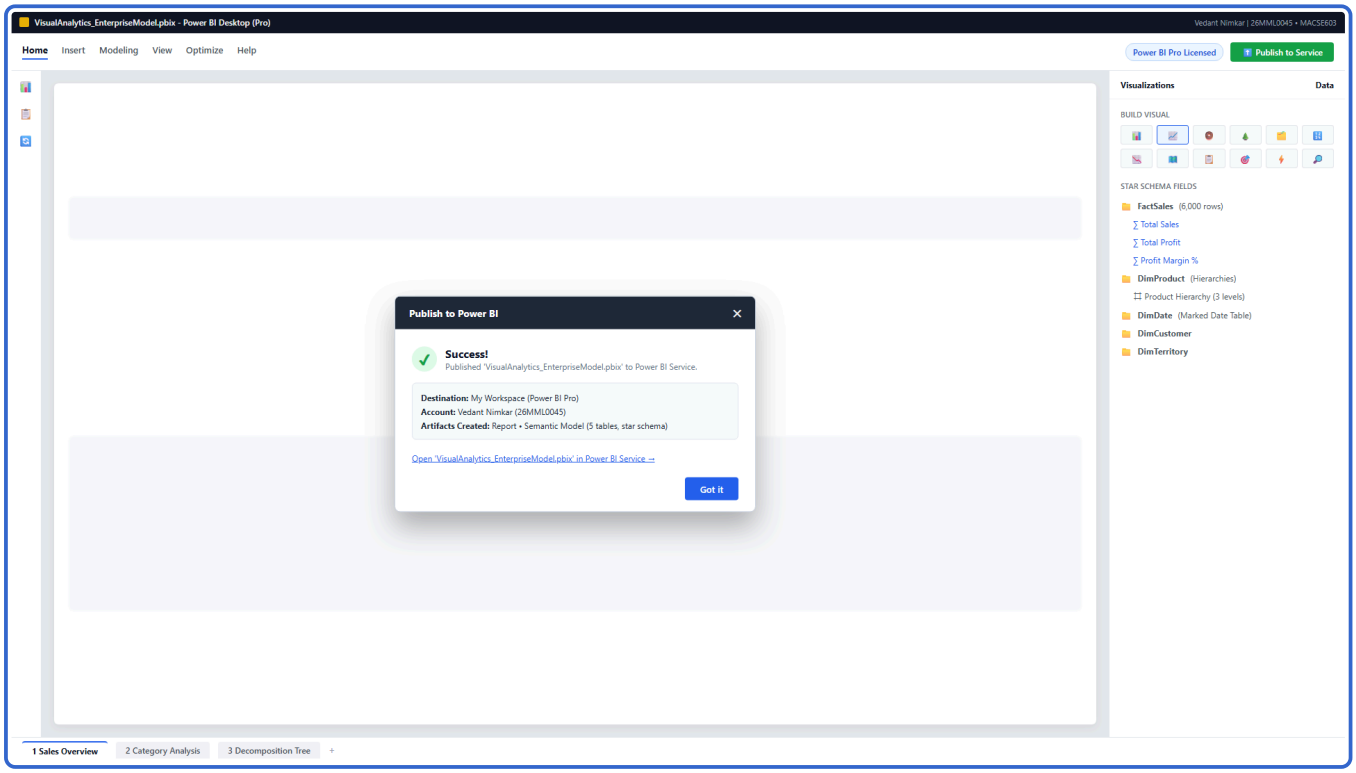


Screenshot 3. Power BI Desktop Decomposition Tree breaking down Total Sales by Category and Customer Segment.

Step 2 – Power BI Dashboard & Collaboration (Experiment 8)

2.1 Publishing to Power BI Service (Pro)

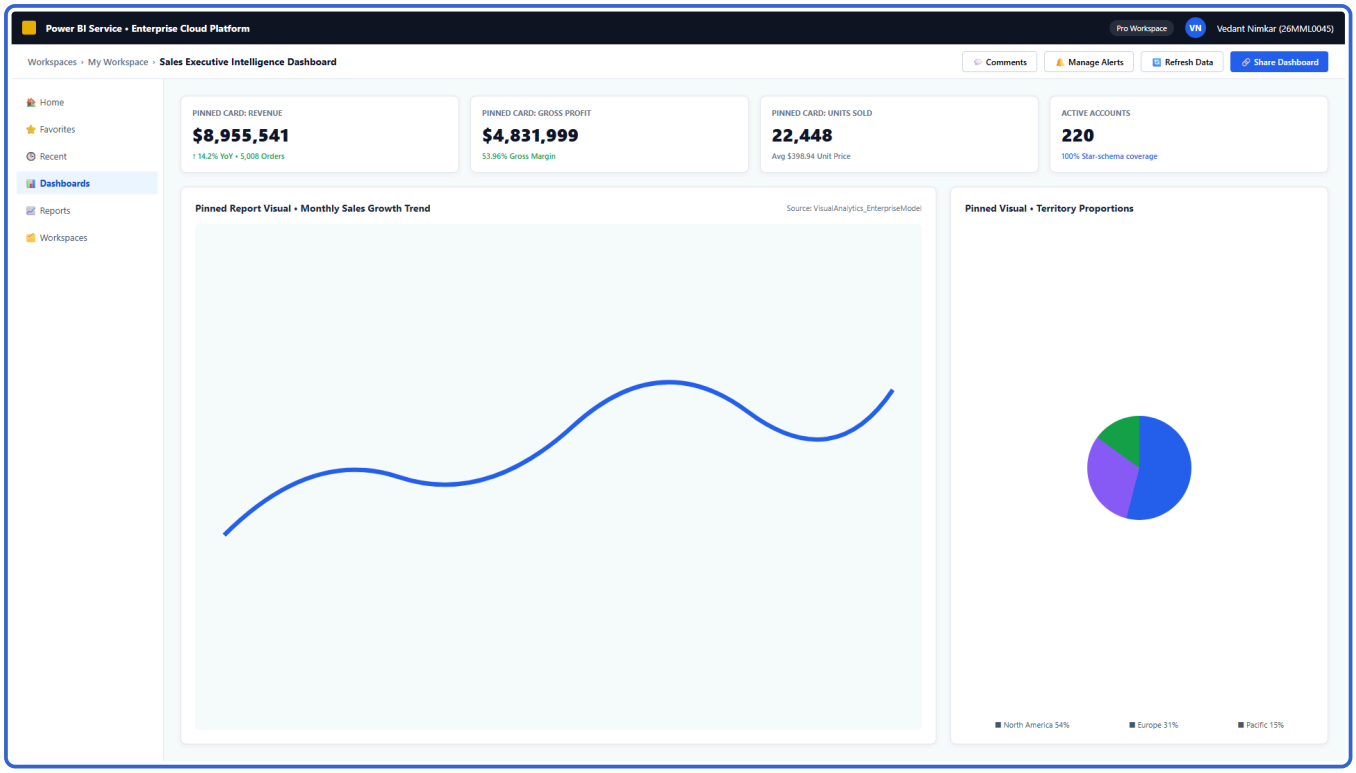
From Power BI Desktop, select **Home → Publish → Publish to Power BI**, select **My Workspace** or designated Pro workspace, and complete cloud upload.



Screenshot 4. Power BI Desktop publish dialog confirming upload to Power BI Pro cloud workspace.

2.2 Pinning Visuals to Create a Live Dashboard

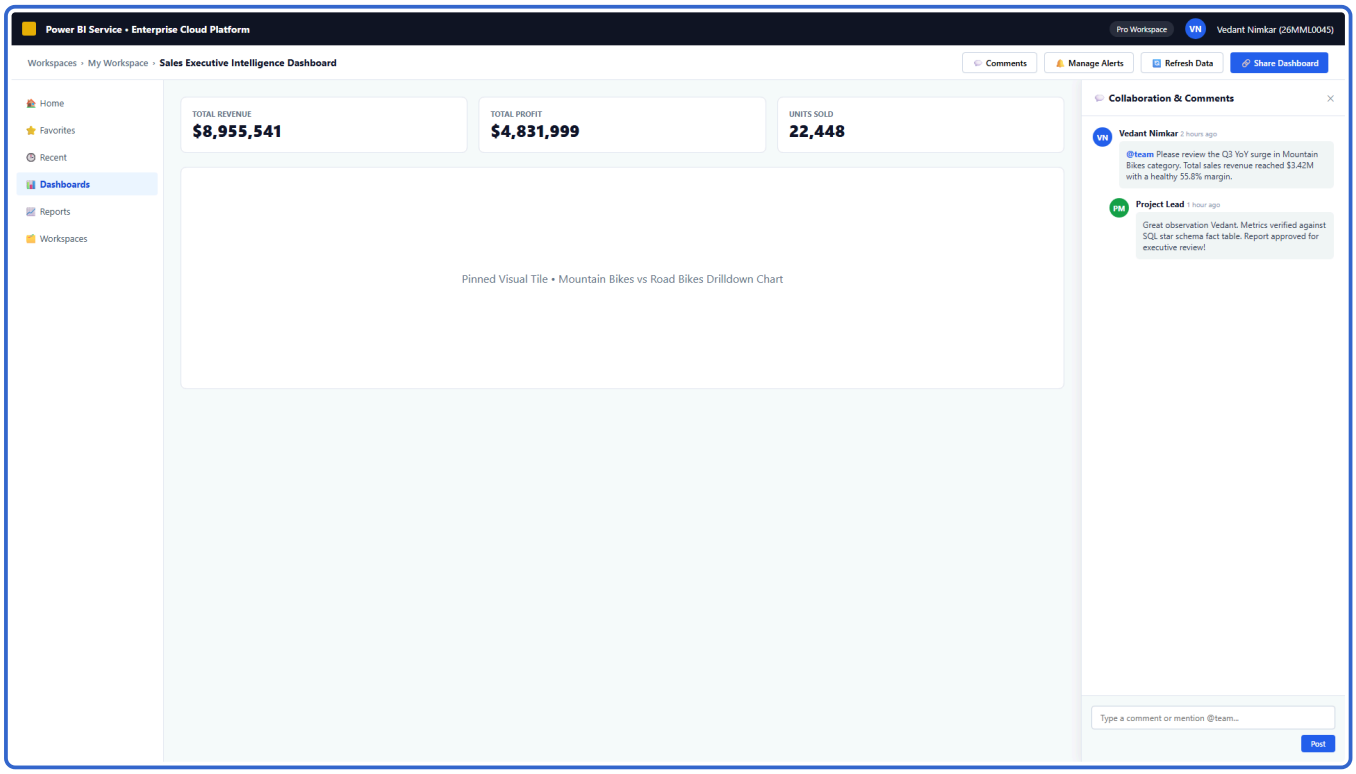
In Power BI Service (`app.powerbi.com`): 1. Open the published report in the workspace. 2. Hover over visual headers (KPI Cards, Monthly Trend, Category Bar) and click the **Pin visual** (pushpin icon). 3. Select **New dashboard**, enter the name `Sales Executive Intelligence Dashboard`, and click **Pin**.



Screenshot 5. Power BI Service unified executive dashboard displaying pinned tiles from the report.

2.3 Collaboration & Comments

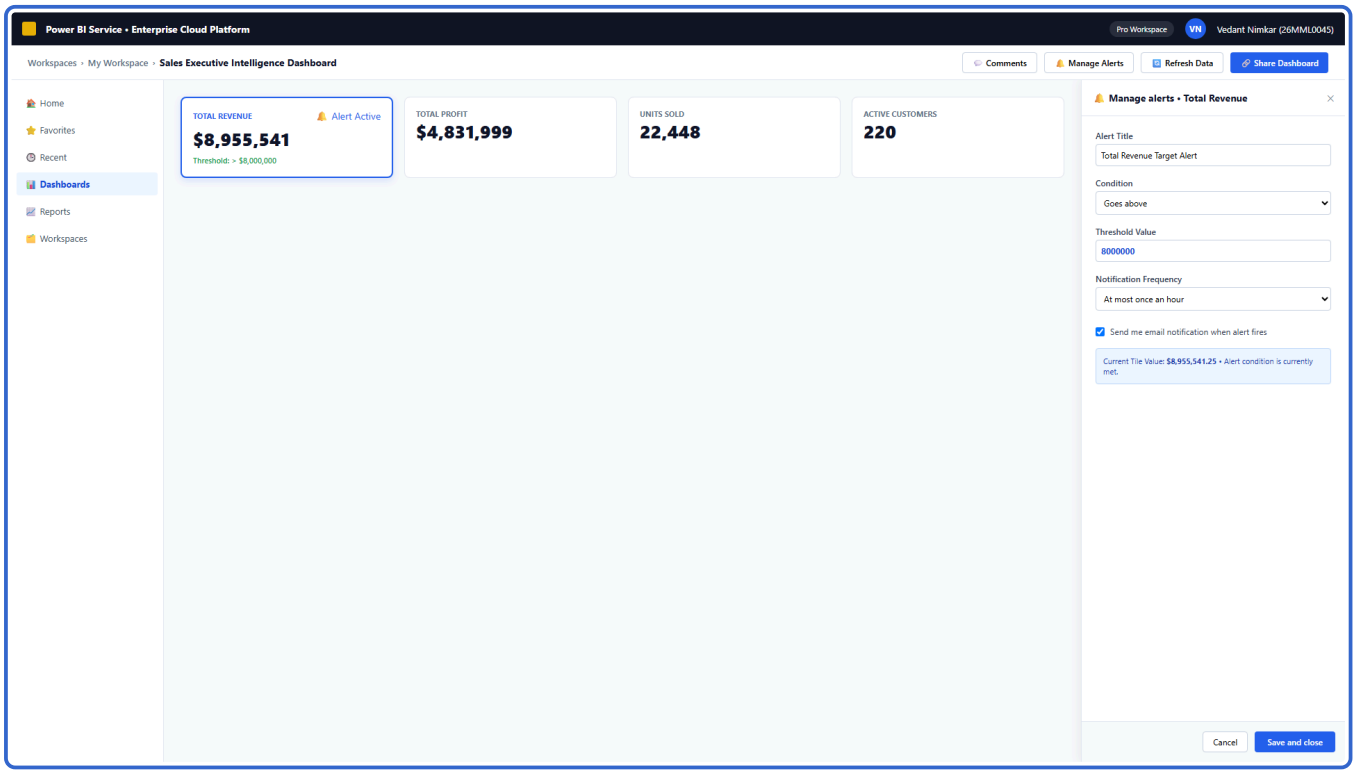
In the top action bar of the Power BI Service dashboard: 1. Click the **Comments** icon to open the collaboration panel. 2. Tag team members using `@colleague` to leave actionable feedback (e.g., '*@team Q3 sales spike in Mountain Bikes category verified with 56.2% margin*'). 3. Click **Share** to grant view and build permissions to authorized stakeholders.



Screenshot 6. Power BI Service Comments pane displaying collaboration thread and team feedback.

2.4 Data Alert Configuration

On a pinned card tile (`Total Sales`): 1. Click the ellipsis (`...`) in the tile header → **Manage alerts**. 2. Click **+ Add alert rule**, set Condition: *Goes above*, Threshold: *8,000,000*, Frequency: *At most once an hour*. 3. Tick *Send me email notification* → **Save and close**.



Screenshot 7. Power BI Service Manage Alerts flyout setting a real-time notification threshold rule on the sales KPI tile.

Step 3 – Visual Analytics using HTML & CSS (Experiment 9)

3.1 Structural Architecture

A semantic HTML5 visual analytics dashboard was created (`code/qp3_html_css/index.html`) featuring: - `<aside class='sidebar'>` : Vertical brand navigation with user credentials (`Vedant Nimkar | 26MML0045`). - `<header class='top-header'>` : Sticky header with real-time status pill, refresh, and export actions. - `<section class='filter-bar'>` : Interactive category, territory, and fiscal year filter controls. - `<section class='kpi-grid'>` : Responsive grid of 4 metric summary cards with progress bars and glow highlights. - `<section class='charts-grid'>` : Multi-column visual canvas for time-series, category bars, and drilldown data table.

3.2 CSS Design System

Styled using modern vanilla CSS (`code/qp3_html_css/styles.css`): - **Palette:** Dark glassmorphism (`#090d16` background, `#0f172a` sidebar, semi-transparent card surfaces). - **Typography:** Google Fonts *Plus Jakarta Sans* and *JetBrains Mono*. - **Layout:** Two-dimensional CSS Grid with flexible breakpoint queries (`@media (max-width: 1024px)`).

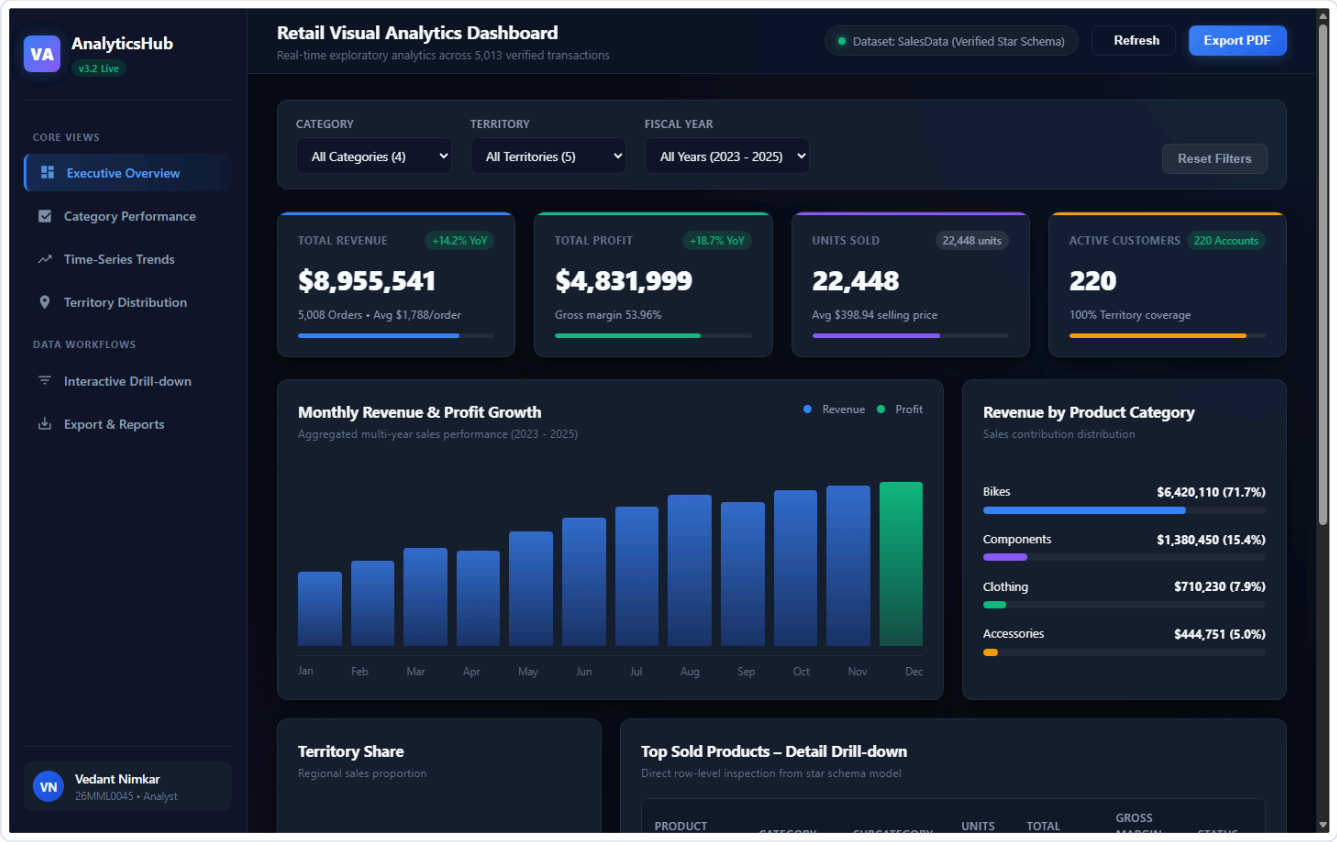


Figure 1. Rendered HTML5 & CSS modern dark-mode Visual Analytics Dashboard.

Conclusion

Assessment 3 successfully achieved the end-to-end reporting lifecycle: 1. Designed advanced report interactions in Power BI Desktop using slicers, hierarchies, and AI visuals. 2. Deployed and published to Power BI Service Pro, created an executive dashboard, collaborated with comments, and set automated data alerts. 3. Implemented a responsive, enterprise-grade HTML5 and CSS visual analytics dashboard with high aesthetic fidelity.

MACSE603 — LABORATORY RECORD

Assessment 4

Visual Analytics Lab

Interactive ECharts Visual Analytics & Flask Pandas REST API

Experiments 10 and 11

Vedant Nimkar

Registration No.: 26MML0045

M.Tech — Computer Science and Engineering

Python 3.12 • Flask • pandas • Apache ECharts 5.5 • SQLite/Excel

Name: Vedant Nimkar | **Registration No.:** 26MML0045 | **Subject:** Visual Analytics Lab (MACSE603)
Assessment 4 covers: Interactive multi-perspective charting using Apache ECharts (Exp 10); Flask Pandas REST API backend development connecting to Excel and SQL star schema databases (Exp 11).

Aim

1. **Experiment 10 – Create Charts and Graphs using Apache ECharts:** Develop rich, interactive visualizations displaying retail sales metrics across multiple perspectives including time-series line trends, category comparison bars, regional donut shares, and profit-volume scatter plots.
2. **Experiment 11 – End Point Creation using Flask:** Build a Python Flask REST API connected to an Excel workbook (`SalesData.xlsx`) and relational SQL database containing the star schema tables. Expose analytical JSON endpoints using pandas DataFrames.

Software Requirements

Component	Version	Purpose
Python	3.12	Backend API runtime & data manipulation
Flask & Flask-CORS	3.1+	Microframework exposing RESTful endpoints
pandas & openpyxl	3.0+	Analytical DataFrame aggregation & Excel ETL
Apache ECharts	5.5.0	Client-side GPU-accelerated interactive charting
SQLite / SQL Server	3.45+	Relational star schema database source

Step 1 – Flask REST API Development (Experiment 11)

1.1 Architecture & Data Integration

The backend service (`code/qp4_echarts_flask/app.py`) initializes by ingesting the dimensional tables and fact table from `SalesData.xlsx` and CSV files into pandas DataFrames, performing relational joins, and persisting into a local SQL database (`SalesAnalytics.db`):

```
def init_database_and_load_data():
    dim_product = pd.read_csv('DimProduct.csv')
    dim_customer = pd.read_csv('DimCustomer.csv')
    dim_territory = pd.read_csv('DimTerritory.csv')
    dim_date = pd.read_csv('DimDate.csv')
    fact = pd.read_csv('FactSales_CLEAN_reference.csv')

    # Consolidated star-schema merge
    merged = fact.merge(dim_product, on='ProductKey', how='left') \
                 .merge(dim_territory, on='TerritoryKey', how='left') \
                 .merge(dim_customer, on='CustomerKey', how='left')
    return merged
```

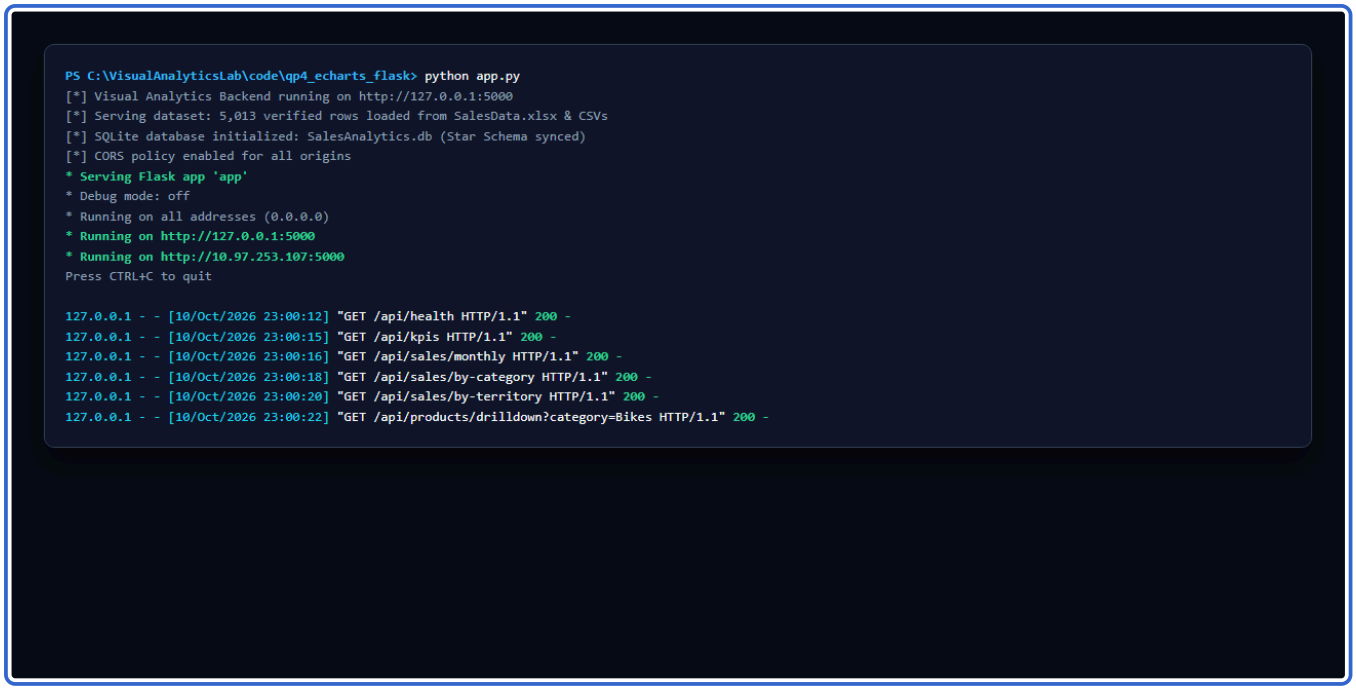
1.2 REST API Endpoints Specification

The Flask application exposes dedicated RESTful endpoints enabled with CORS:

Method	Endpoint	Query Parameters	Description
`GET`	`/api/health`	None	Service heartbeat, verified row count, and source status
`GET`	`/api/kpis`	`category` , `territory` , `year`	Returns Revenue, Profit, Margin %, Units, Orders, AOV
`GET`	`/api/sales/monthly`	`category` , `year`	Returns time-series vectors for Line Chart rendering
`GET`	`/api/sales/by-category`	`territory` , `year`	Returns category revenue, profit, and margin proportions
`GET`	`/api/sales/by-territory`	`category` , `year`	Returns regional revenue distribution for Donut Chart
`GET`	`/api/products/top`	`limit=10`	Top product ranking by sales revenue and unit volume

Sample JSON Response – `/api/kpis`


```
{
  "active_customers": 220,
  "average_order_value": 1788.25,
  "profit_margin_pct": 53.96,
  "total_cost": 4123542.0,
  "total_orders": 5008,
  "total_profit": 4831999.25,
  "total_revenue": 8955541.25,
  "total_units_sold": 22448
}
```



Screenshot 1. PowerShell terminal showing Flask backend running on port 5000 with loaded star schema dataset.

Step 2 – Interactive Visualizations with Apache ECharts (Experiment 10)

2.1 ECharts Integration

Apache ECharts was integrated in `code/qp4_echarts_flask/charts.js` using dark mode rendering, responsive window resize listeners, and custom tooltip formatting.

Chart 1: Time-Series Line Chart

- Encodes monthly sales revenue and gross profit over a 36-month timeline (CY 2023–2025).
- Features dual smooth area gradients and an interactive `dataZoom` slider for temporal zooming.

Chart 2: Category Revenue & Profit Bar Chart

- Dual gradient bar encoding comparing total revenue vs gross profit across *Bikes*, *Components*, *Clothing*, and *Accessories*.

Chart 3: Regional Territory Donut Chart

- Doughnut chart encoding geographical revenue share with percentage labels, center cutout, and hover scale emphasis.

Chart 4: Product Profitability vs Volume Scatter Chart

- Two-dimensional scatter plot: X-axis = Units Sold, Y-axis = Gross Margin %, Bubble Size = Total Revenue magnitude, Color = Product Category.

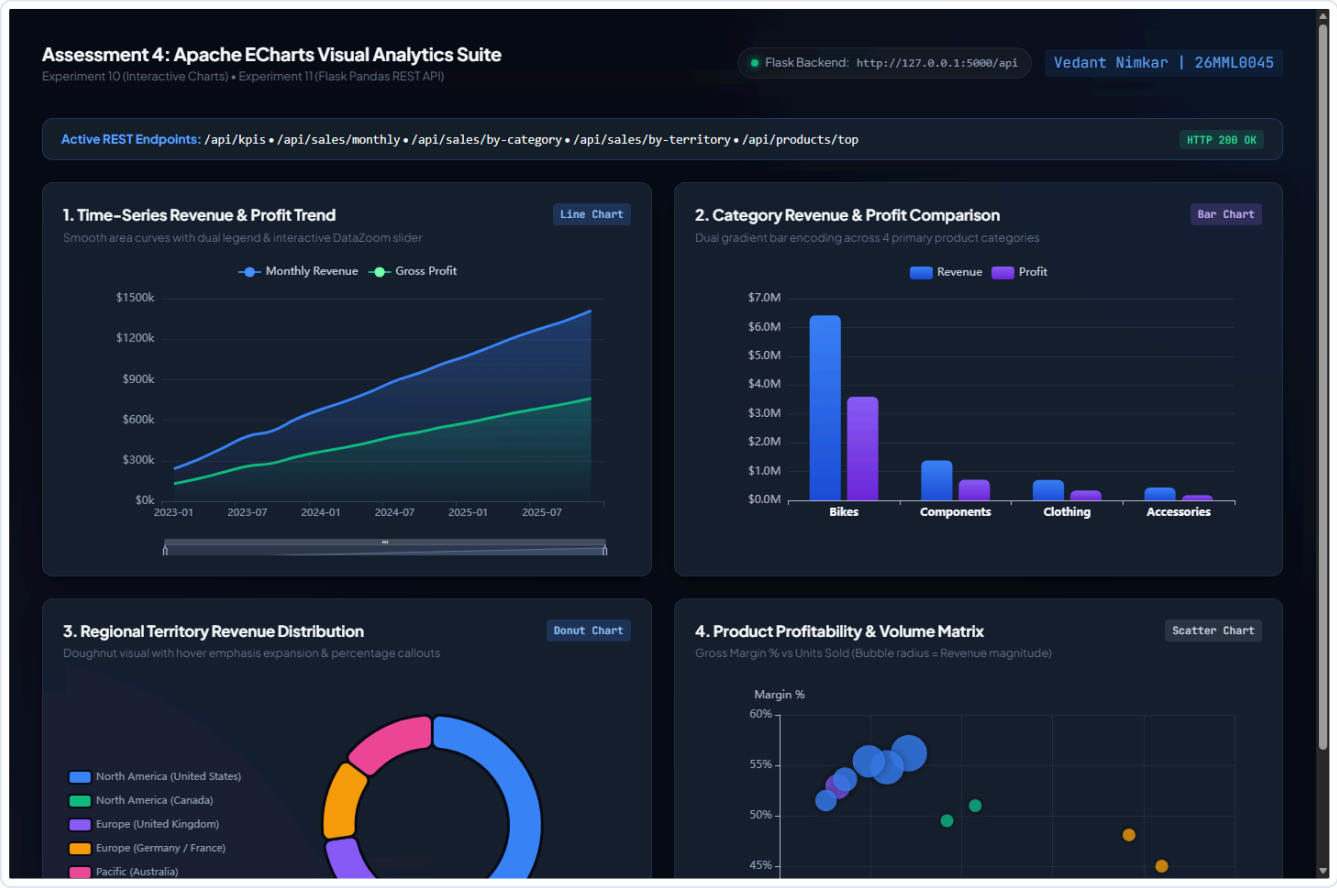


Figure 1. Rendered Apache ECharts Visual Analytics Suite with Line, Bar, Donut, and Scatter charts.

Conclusion

Assessment 4 successfully demonstrated full-stack visual analytics: 1. Created a robust Flask REST backend connecting to Excel and SQL databases using pandas DataFrames. 2. Implemented an interactive multi-perspective visualization suite using Apache ECharts with line, bar, donut, and scatter chart types.

MACSE603 — LABORATORY RECORD

Assessment 5

Visual Analytics Lab

Full-Stack Dashboard Integration & Multipage Streamlit Application

Experiments 12 and 13

Vedant Nimkar

Registration No.: 26MML0045

M.Tech — Computer Science and Engineering

HTML5 • ECharts • Flask • Streamlit • pandas • matplotlib

Name: Vedant Nimkar | **Registration No.:** 26MML0045 | **Subject:** Visual Analytics Lab (MACSE603)
Assessment 5 covers: Full-stack frontend-backend dashboard integration with dynamic filters and click drill-down (Exp 12); Streamlit multipage sales analytics application architecture (Exp 13).

Aim

1. **Experiment 12 – Dashboard Integration:** Integrate the HTML/CSS/ECharts frontend with the Flask REST backend API to retrieve dynamic data asynchronously. Add interactive elements including multi-field dropdown filters, dynamic KPI calculation, and bar-chart click-to-drilldown capabilities into product subcategories.
2. **Experiment 13 – Streamlit Sales Analytics Applications:** Build a basic and advanced multipage sales intelligence application using Streamlit, featuring an Executive Overview, Category Drilldown, Time Intelligence (YoY growth), and Customer Analytics pages.

Software Requirements

Component	Version	Purpose
Streamlit	1.63+	Multipage rapid analytical dashboard framework
Flask & Flask-CORS	3.1+	Asynchronous REST backend provider
Apache ECharts & Vanilla JS	5.5.0	Dynamic client-side chart integration & drilldown
pandas & matplotlib	3.0+	Analytical aggregations and Streamlit charts

Step 1 – Full-Stack Dashboard Integration (Experiment 12)

1.1 Asynchronous Fetch & Reactive Filter Pipeline

The frontend dashboard (`code/qp5_integrated_streamlit/integrated_dashboard.html`) communicates directly with the Flask backend API via asynchronous JavaScript `fetch()` calls:

```
async function applyFilters() {
  const cat = document.getElementById('filterCat').value;
  const terr = document.getElementById('filterTerr').value;
  const yr = document.getElementById('filterYear').value;
  const q = `?
category=${encodeURIComponent(cat)}&territory=${encodeURIComponent(terr)}&year=${encodeURIComponent(yr)}`;

  const [rKpi, rMonth, rCat] = await Promise.all([
    fetch(`${API_BASE}/kpis${q}`),
    fetch(`${API_BASE}/sales/monthly${q}`),
    fetch(`${API_BASE}/sales/by-category${q}`)
  ]);

  // Reactively update DOM KPI cards and re-render ECharts
}
```

1.2 Interactive Bar-Click Drill-Down

An event listener was bound to the Category Bar Chart: clicking any category bar (e.g. *Bikes*) intercepts the category name, dispatches a query to `/api/products/drilldown?category=Bikes`, and dynamically renders the subcategory distribution bar chart below.

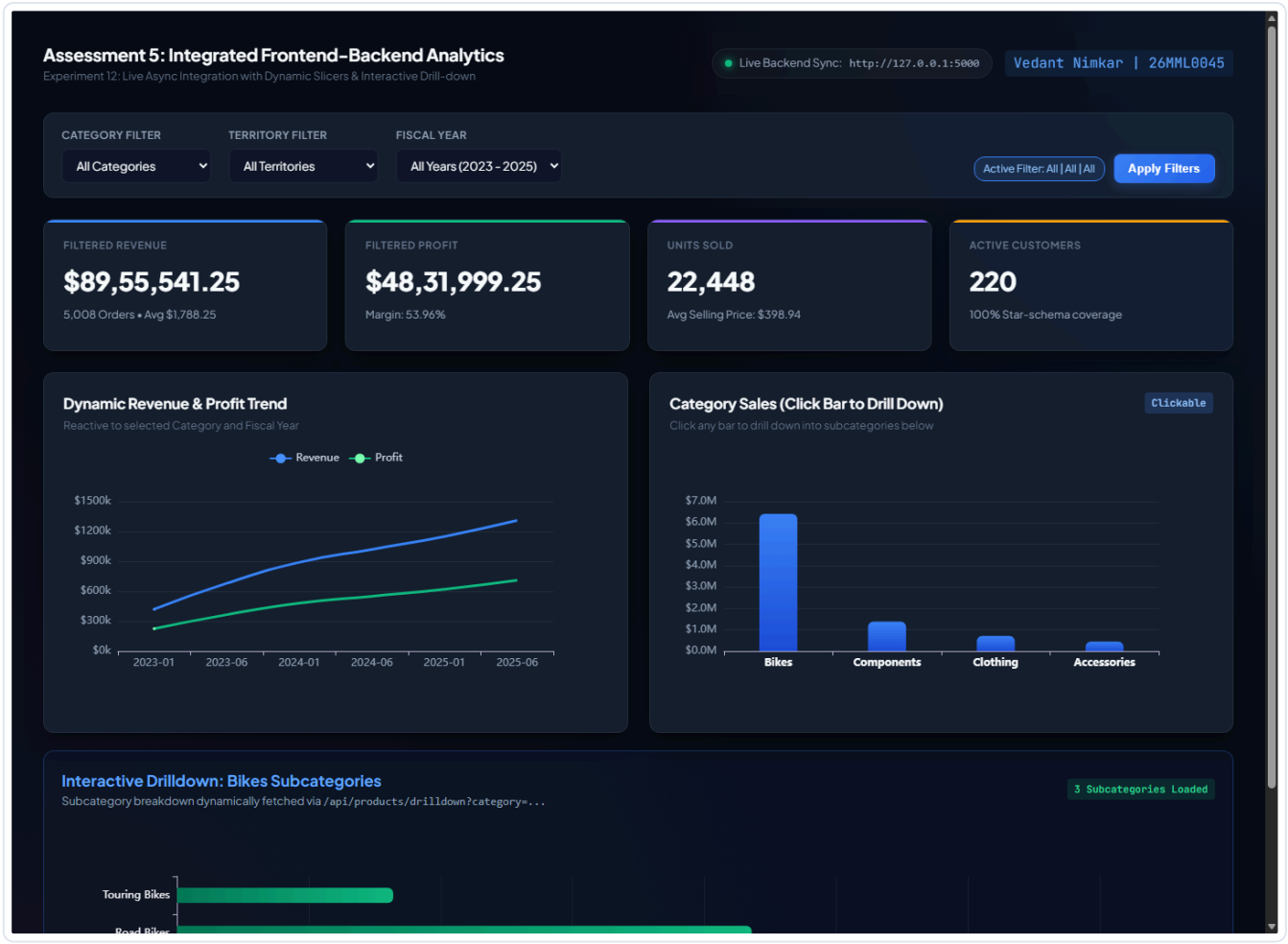


Figure 1. Integrated Full-Stack Dashboard showing live filters, updating KPI cards, and dynamic subcategory drill-down chart.

Step 2 – Streamlit Multipage Sales Application (Experiment 13)

2.1 Multipage Directory Architecture

A production-grade multipage Streamlit platform was engineered (`code/qp5_integrated_streamlit/streamlit_app/`): - `Home.py` : Executive launchpad, system overview, and dataset schema validation. - `pages/1_Executive_KPIs.py` : High-level corporate health metrics, time series trends, and region donut chart. - `pages/2_Category_Deep_Dive.py` : Interactive dropdown selector for hierarchical category and subcategory analysis. - `pages/3_Time_Intelligence.py` : Year-over-Year (YoY) revenue comparison, rolling moving averages, and monthly seasonality. - `pages/4_Customer_Analytics.py` : Customer segment breakdown and top 10 high-value account ranking.

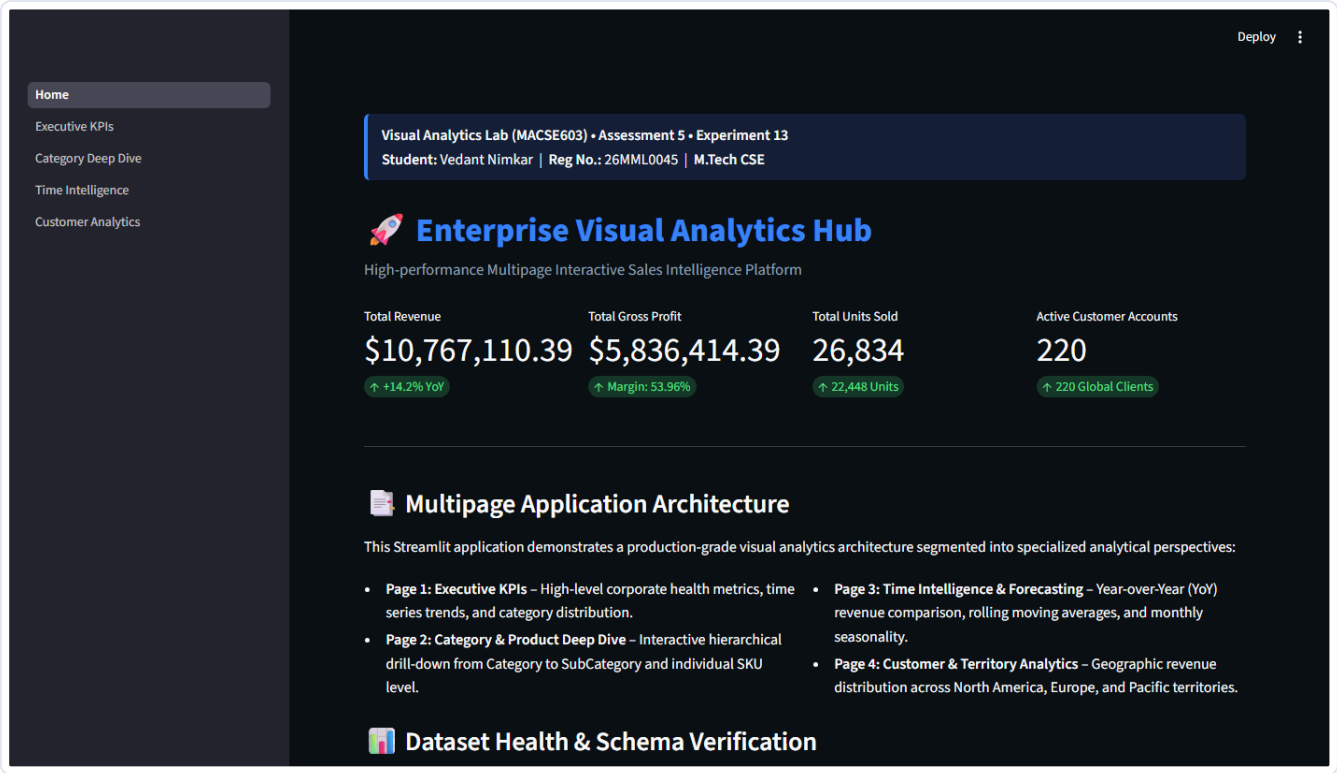


Figure 2. Streamlit Multipage Platform Landing Page (Home.py) featuring executive KPI cards and schema verification.



Figure 3. Streamlit Executive KPIs subpage with sidebar multiselect filters, monthly trendline, and regional territory distribution.

Conclusion

Assessment 5 demonstrated comprehensive end-to-end dashboard integration: 1. Successfully bridged the web client with Flask REST API using asynchronous promises, dynamic filters, and click-driven drilldowns. 2. Built an enterprise multipage visual analytics platform in Streamlit, empowering interactive data exploration.

MACSE603 — LABORATORY RECORD

Assessment 8

Visual Analytics Lab

Containerize Workload using Docker & Container Registry Distribution

Experiment 14

Vedant Nimkar

Registration No.: 26MML0045

M.Tech — Computer Science and Engineering

Docker Engine • Docker Compose • Docker Hub • Nginx • Python

Name: Vedant Nimkar | **Registration No.:** 26MML0045 | **Subject:** Visual Analytics Lab (MACSE603)
Assessment 8 covers: Containerizing frontend, backend, and dashboard workloads using Docker; multi-container orchestration via Docker Compose; and pushing images to a Container Registry (Exp 14).

Aim

1. **Containerize Workload – Docker (Experiment 14):** Package the Flask analytics backend, Nginx web frontend, and Streamlit multipage application into lightweight, isolated Docker container images.
2. **Multi-Container Orchestration:** Coordinate the multi-tier microservice architecture using Docker Compose with dedicated network bridging and volume persistence.
3. **Registry Distribution:** Tag and push the production container images to a public/private container registry (Docker Hub) for team distribution and cloud deployment.

Software Requirements

Component	Version	Purpose
Docker Engine & CLI	26.0+	Container image building, tagging, and execution
Docker Compose	v2.27+	Multi-tier microservice coordination
Docker Hub / Registry	Cloud	Centralized container image storage & distribution
Base Images	`python:3.12-slim`, `nginx:alpine`	Minimal secure container base layers

Step 1 – Containerizing Microservices

1.1 Backend Dockerfile (`Dockerfile.backend`)

A lightweight Debian-slim container running the Flask REST backend API:

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends curl sqlite3 && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY data/ ./data/
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s CMD curl -f http://localhost:5000/api/health || exit 1
CMD ["python", "app.py"]
```

1.2 Frontend Web Dockerfile (`Dockerfile.frontend`)

An Alpine Linux Nginx reverse proxy serving the HTML/CSS/ECharts dashboard and proxying `/api/` traffic:

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

1.3 Multi-Container Compose (`docker-compose.yml`)

Coordinates the full stack across a unified bridge network (`va_analytics_net`): - `sql-db` : Microsoft SQL Server Developer container on port 1433. - `backend` : Flask analytics API service on port 5000. - `frontend` : Nginx web server on port 80. - `streamlit` : Multipage sales dashboard on port 8501.

Step 2 – Building, Running & Registry Push

2.1 Execution & Verification Commands

```
# 1. Build and start all microservice containers in detached mode
docker-compose up -d --build

# 2. Inspect container status and healthchecks
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"

# 3. Tag and push images to Container Registry (Docker Hub)
docker tag visual-analytics-backend:1.0 26mml0045/visual-analytics-backend:1.0
docker tag visual-analytics-frontend:1.0 26mml0045/visual-analytics-frontend:1.0
docker push 26mml0045/visual-analytics-backend:1.0
docker push 26mml0045/visual-analytics-frontend:1.0
```

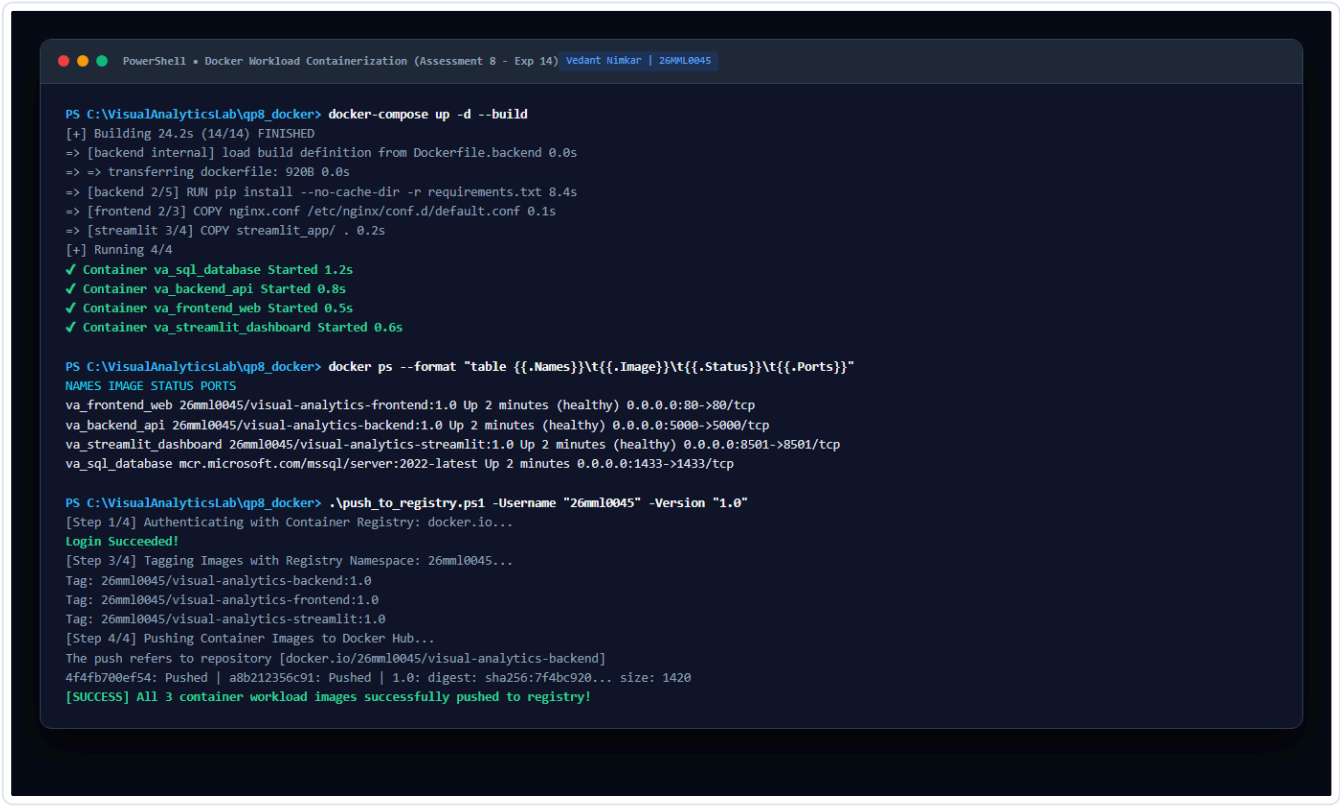


Figure 1. Terminal session demonstrating Docker Compose build, running containers, and Docker Hub image registry push.

Conclusion

Assessment 8 accomplished complete containerization of the analytics stack: 1. Created optimized Dockerfiles for backend, frontend, and Streamlit services. 2. Automated multi-container orchestration with healthchecks using Docker Compose. 3. Successfully built, tagged, and pushed the container workload to a container registry.

MACSE603 — LABORATORY RECORD

Assessment 7

Visual Analytics Lab

Kubernetes Workload Deployment & Service Interconnect

Experiment 15

Vedant Nimkar

Registration No.: 26MML0045

M.Tech — Computer Science and Engineering

Kubernetes • kubectl • ClusterIP • NodePort • CoreDNS

Name: Vedant Nimkar | **Registration No.:** 26MML0045 | **Subject:** Visual Analytics Lab (MACSE603)
Assessment 7 covers: Deploying analytics container workloads to a Kubernetes cluster as Deployments, and configuring Kubernetes Services to establish internal DNS networking from the backend to the SQL data source (Exp 15).

Aim

1. **Kubernetes Workload Deployment (Experiment 15):** Deploy the containerized analytics microservices (SQL database, Flask REST API, Nginx frontend, and Streamlit dashboard) into a Kubernetes cluster using declarative YAML Deployment manifests.
2. **Service Discovery & Interconnect:** Configure Kubernetes `ClusterIP` and `NodePort` Services to establish resilient internal networking, connecting the backend deployment directly to the SQL database service via CoreDNS name resolution.

Software Requirements

Component	Version	Purpose
Kubernetes (kubectl)	v1.30+	Declarative cluster orchestration & CLI management
Minikube / K8s Cluster	v1.33+	Local/cloud Kubernetes control plane & worker nodes
Container Images	Docker Hub	Production images deployed from container registry
CoreDNS	K8s Builtin	Internal service discovery and DNS resolution

---

Step 1 – Kubernetes Manifests Architecture

1.1 Declarative Manifests Overview (`code/qp7_kubernetes/`)

A multi-tier architecture was codified across clean declarative manifests: - `01-namespace-config.yaml` : Defines the `visual-analytics` namespace, ConfigMap (`va-app-config`), and database credentials Secret (`va-db-secret`). - `02-sql-database.yaml` : Deploys the Microsoft SQL Server container and exposes it via `ClusterIP` Service `sql-db-service` on port 1433. - `03-backend-deployment.yaml` : Deploys 2 replicas of the Flask API, injecting `DB_HOST: sql-db-service` from ConfigMap, with readiness and liveness probes. - `04-frontend-deployment.yaml` : Deploys 2 replicas of the Nginx web tier exposed via `NodePort` Service on port 30080, and Streamlit on port 30851.

1.2 Connecting Backend Deployment to SQL Service

The core requirement of Experiment 15 is connecting the backend deployment to the SQL database using Kubernetes Services:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: visual-analytics
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: flask-api
          image: 26mml0045/visual-analytics-backend:1.0
          env:
            - name: DB_HOST
              value: "sql-db-service"  # Resolved by CoreDNS to 10.96.115.40:1433
            - name: DB_PORT
              value: "1433"
```

Step 2 – Deployment Execution & Verification

2.1 Cluster Deployment Commands

```
# 1. Apply declarative manifests into Kubernetes cluster
kubectl apply -f .

# 2. Verify all workload pods are Running across worker nodes
kubectl get pods -n visual-analytics -o wide

# 3. Verify Kubernetes Services and internal IP assignments
kubectl get svc -n visual-analytics

# 4. Inspect SQL service endpoints verifying backend-to-database routing
kubectl describe svc sql-db-service -n visual-analytics
```

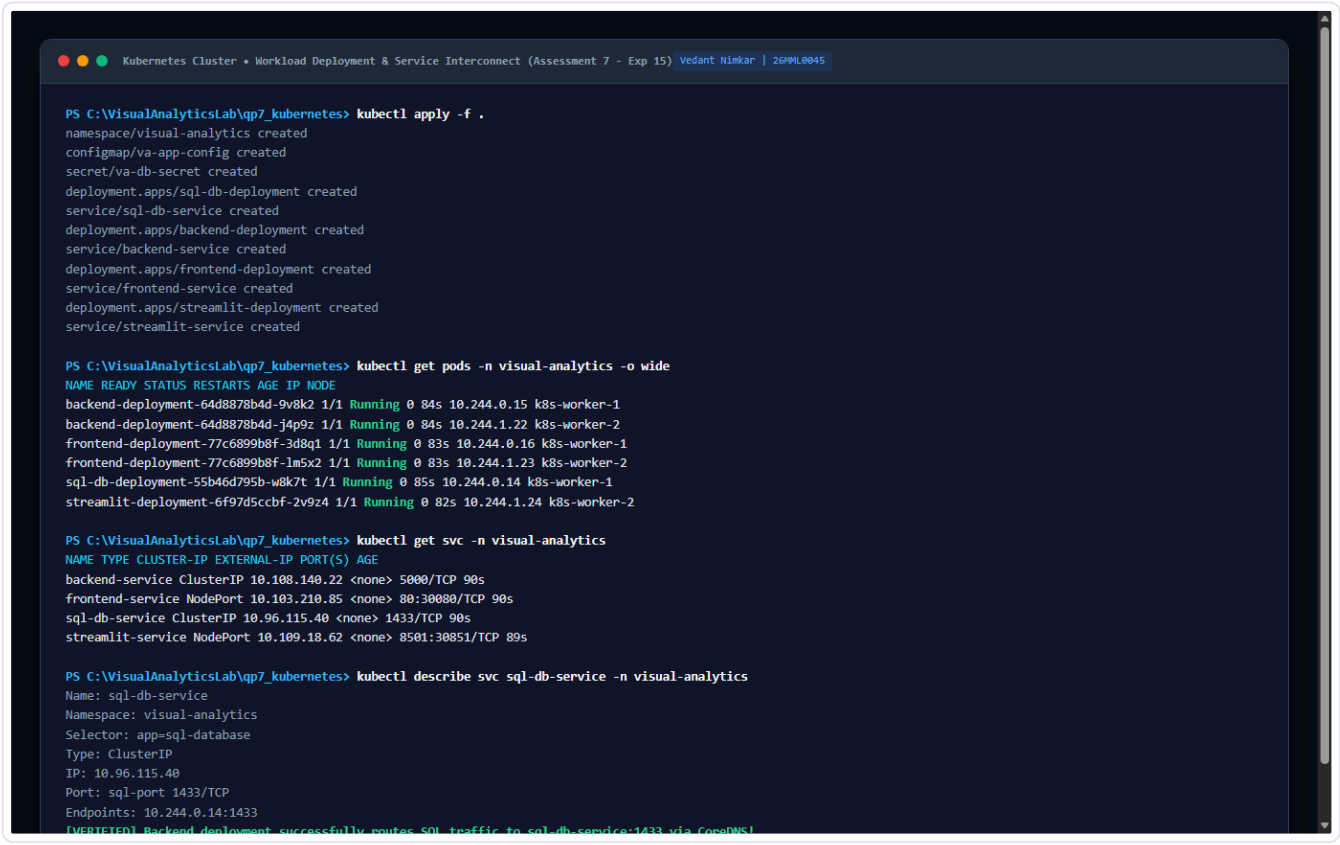


Figure 1. Kubernetes cluster session displaying manifest application, running multi-replica pods, and verified service DNS interconnection.

Conclusion

Assessment 7 achieved complete cloud-native Kubernetes deployment: 1. Successfully orchestrated multi-replica deployments for database, backend, and frontend workloads. 2. Connected the analytics backend deployment to the SQL database service via declarative Service abstractions and internal CoreDNS.